llnl_c1/lulesh · lulesh.cc · bare_llm_v4 · claude-haiku-4-5-20251001 · FASTER

−9 / +75 lines of 1967 · 65 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (10 code line(s) added, 9 removed) · restructuring level 0

BEFORE

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     for (Index_t i = 0 ; i < numElem ; ++i){
115        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
116     }
117  }
118
     …
308
309     Real_t fx_local[8] ;
310     Real_t fy_local[8] ;
311     Real_t fz_local[8] ;
312
313    for( Index_t k=0 ; k<numElem ; ++k )
314    {
315      const Index_t* const elemToNode = domain.nodelist(k);
316      Real_t B[3][8] ;
317      Real_t x_local[8] ;
     …
329      SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
330                                   fx_local, fy_local, fz_local ) ;
331
332      for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
333        Index_t gnode = elemToNode[lnode];
334        domain.fx(gnode) += fx_local[lnode];
335        domain.fy(gnode) += fy_local[lnode];
336        domain.fz(gnode) += fz_local[lnode];
337      }
338
339    }
340
     …
495    gamma[3][4] = Real_t( 1.);
496    gamma[3][5] = Real_t(-1.);
497    gamma[3][6] = Real_t( 1.);
498    gamma[3][7] = Real_t(-1.);
499
500    for(Index_t i2=0;i2<numElem;++i2){
501       Real_t hgfx[8], hgfy[8], hgfz[8] ;
502
503       Real_t coefficient;
504
     …
607
608       CalcElemFBHourglassForce(xd1,yd1,zd1,
609                       hourgam,
610                       coefficient, hgfx, hgfy, hgfz);
611
612       domain.fx(n0si2) += hgfx[0];
613       domain.fy(n0si2) += hgfy[0];
614       domain.fz(n0si2) += hgfz[0];
615
616       domain.fx(n1si2) += hgfx[1];
617       domain.fy(n1si2) += hgfy[1];
618       domain.fz(n1si2) += hgfz[1];
619
620       domain.fx(n2si2) += hgfx[2];
621       domain.fy(n2si2) += hgfy[2];
622       domain.fz(n2si2) += hgfz[2];
623
624       domain.fx(n3si2) += hgfx[3];
625       domain.fy(n3si2) += hgfy[3];
626       domain.fz(n3si2) += hgfz[3];
627
628       domain.fx(n4si2) += hgfx[4];
629       domain.fy(n4si2) += hgfy[4];
630       domain.fz(n4si2) += hgfz[4];
631
632       domain.fx(n5si2) += hgfx[5];
```

AFTER

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114  #pragma omp parallel for
115     for (Index_t i = 0 ; i < numElem ; ++i){
116        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
117     }
118  }
119
     …
309
310     Real_t fx_local[8] ;
311     Real_t fy_local[8] ;
312     Real_t fz_local[8] ;
313
314  #pragma omp parallel for private(fx_local, fy_local, fz_local)
315    for( Index_t k=0 ; k<numElem ; ++k )
316    {
317      const Index_t* const elemToNode = domain.nodelist(k);
318      Real_t B[3][8] ;
319      Real_t x_local[8] ;
     …
331      SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
332                                   fx_local, fy_local, fz_local ) ;
333
334      for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
335        Index_t gnode = elemToNode[lnode];
336  #pragma omp atomic
337        domain.fx(gnode) += fx_local[lnode];
338  #pragma omp atomic
339        domain.fy(gnode) += fy_local[lnode];
340  #pragma omp atomic
341        domain.fz(gnode) += fz_local[lnode];
342      }
343
344    }
345
     …
500    gamma[3][4] = Real_t( 1.);
501    gamma[3][5] = Real_t(-1.);
502    gamma[3][6] = Real_t( 1.);
503    gamma[3][7] = Real_t(-1.);
504
505  #pragma omp parallel for
506     for(Index_t i2=0;i2<numElem;++i2){
507        Real_t hgfx[8], hgfy[8], hgfz[8] ;
508
509        Real_t coefficient;
510
     …
613
614        CalcElemFBHourglassForce(xd1,yd1,zd1,
615                        hourgam,
616                        coefficient, hgfx, hgfy, hgfz);
617
618  #pragma omp atomic
619        domain.fx(n0si2) += hgfx[0];
620  #pragma omp atomic
621        domain.fy(n0si2) += hgfy[0];
622  #pragma omp atomic
623        domain.fz(n0si2) += hgfz[0];
624
625  #pragma omp atomic
626        domain.fx(n1si2) += hgfx[1];
627  #pragma omp atomic
628        domain.fy(n1si2) += hgfy[1];
629  #pragma omp atomic
630        domain.fz(n1si2) += hgfz[1];
631
632  #pragma omp atomic
633        domain.fx(n2si2) += hgfx[2];
634  #pragma omp atomic
635        domain.fy(n2si2) += hgfy[2];
636  #pragma omp atomic
637        domain.fz(n2si2) += hgfz[2];
638
639  #pragma omp atomic
640        domain.fx(n3si2) += hgfx[3];
641  #pragma omp atomic
642        domain.fy(n3si2) += hgfy[3];
643  #pragma omp atomic
644        domain.fz(n3si2) += hgfz[3];
645
646  #pragma omp atomic
647        domain.fx(n4si2) += hgfx[4];
648  #pragma omp atomic
649        domain.fy(n4si2) += hgfy[4];
650  #pragma omp atomic
651        domain.fz(n4si2) += hgfz[4];
652
653  #pragma omp atomic
654        domain.fx(n5si2) += hgfx[5];
655  #pragma omp atomic
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is